Il linguaggio SQL
Viste e tabelle derivate

Tutti i diritti riservati

Prof. Fedeli Massimo - Tutti i diritti riservati

Versione elettronica: 04.5.SQL.viste

2026

DB di riferimento per gli esempi

Imp

CodImp	Nome	Sede	Ruolo	Stipendio
E001	Rossi	S01	Analista	2000
E002	Verdi	S01	Sistemista	1500
E003	Bianchi	S01	Programmatore	1000
E004	Gialli	S03	Programmatore	1000
E005	Neri	S02	Analista	2500
E006	Grigi	S01	Sistemista	1100
E007	Violetti	S01	Programmatore	1000
E008	Aranci	S02	Programmatore	1200

Sedi

Sede	Città
S01	Bologna
S02	Milano
S03	Roma

Prog

CodProg	Nome	Città
P01	Alpha	Bologna
P02	Beta	Milano

Definizione di viste

Concetto

Mediante l'istruzione `CREATE VIEW` si definisce una "tabella virtuale".
Le tuple della vista sono il risultato di una query calcolata dinamicamente ogni volta che si fa riferimento alla vista.

```sql
CREATE VIEW ProgSedi(CodProg,CodSede)
AS SELECT P.CodProg, S.Sede
   FROM Prog P, Sedi S
   WHERE P.Citta = S.Citta

SELECT *
FROM ProgSedi
WHERE CodProg = 'P01'
```

Risultato:

CodProg	CodSede
P01	S01
P01	S03
P01	S02

Uso delle viste

Le viste possono essere create a vari scopi, tra cui i seguenti:

- Permettere agli utenti di avere una **visione personalizzata** del DB, che in parte astragga dalla struttura logica del DB stesso
- Far fronte a **modifiche dello schema logico** che causerebbero la ricompilazione dei programmi applicativi
- **Semplificare la scrittura** di query complesse

Nota

Le viste possono essere usate come meccanismo di **controllo degli accessi**, fornendo ad ogni classe di utenti solo i privilegi necessari.

Nella definizione di una vista si possono usare anche **altre viste**.

Indipendenza logica tramite viste

Si consideri un DB con:

`EsamiSIT(Matr, Cognome, Nome, DataProva, Voto)`

Per evitare di ripetere i dati anagrafici, si decide di modificare lo schema, sostituendo alla tabella `EsamiSIT` due tabelle:

- `StudentiSIT(Matr, Cognome, Nome)`
- `ProveSIT(Matr, DataProva, Voto)`

È possibile ripristinare la "visione originale":

```
CREATE VIEW EsamiSIT
  (Matr,Cognome,Nome,
   DataProva,Voto)
AS SELECT S.*, P.DataProva,
          P.Voto
   FROM StudentiSIT S,
        ProveSIT P
   WHERE S.Matr = P.Matr
```

Query complesse che usano viste (1)

Esempio

La sede che ha il massimo numero di impiegati

Soluzione senza viste:

```
SELECT I.Sede
FROM Imp I
GROUP BY I.Sede
HAVING COUNT(*) >= ALL
       (SELECT COUNT(*)
        FROM Imp I1
        GROUP BY I1.Sede)
```

⇒ La soluzione è corretta ma poco leggibile e non intuitiva.

Query complesse che usano viste (2)

Soluzione con viste:

```
CREATE VIEW NumImp(Sede, NImp)
AS SELECT Sede, COUNT(*)
   FROM Imp
   GROUP BY Sede

SELECT Sede
FROM NumImp
WHERE NImp = (SELECT MAX(NImp)
              FROM NumImp)
```

Osservazione

Questa formulazione permette di trovare "il MAX dei COUNT(*)", operazione che non si può fare direttamente scrivendo `MAX(COUNT(*))`.

Query complesse che usano viste (3)

Con le viste è possibile risolvere query che richiedono **più passi di raggruppamento**:

Esempio

Per ogni valore (arrotondato) di stipendio medio per sede, il numero delle sedi che pagano tale stipendio medio

Occorre aggregare prima per sede, poi per valore di stipendio medio:

```
CREATE VIEW StipSedi(Sede, AvgStip)
AS SELECT Sede, AVG(Stipendio)
   FROM Imp
   GROUP BY Sede

SELECT AvgStip, COUNT(*) AS NumSedi
FROM StipSedi
GROUP BY AvgStip
```

Aggiornamento di viste

Le viste possono essere utilizzate per le interrogazioni come le normali tabelle del DB, ma per le operazioni di aggiornamento sorgono problemi.

```
CREATE VIEW NumImp(Sede, NImp)
AS SELECT Sede, COUNT(*)
   FROM Imp
   GROUP BY Sede

UPDATE NumImp
SET NImp = NImp + 1
WHERE Sede = 'S03'
```

Problema

Cosa significa incrementare il numero di impiegati di una sede?

Non è possibile tradurre questa operazione in un aggiornamento della tabella `Imp`: non si può fare!

Aggiornabilità di viste (1)

Definizione formale

Una vista è una funzione che calcola un risultato r_V a partire da un'istanza di database r:

$$r_V = V(r)$$

L'aggiornamento di una vista (trasformare r_V in r'_V) è possibile **solo se** è univocamente definita la nuova istanza r' del DB — ovvero se la vista è **invertibile**.

Data la complessità del problema, ogni DBMS definisce quali viste sono aggiornabili.

Restrizioni comuni — il blocco più esterno della query di definizione non deve contenere:

- `GROUP BY`
- Funzioni aggregate
- `DISTINCT`
- Join (espliciti o impliciti)

Aggiornabilità di viste (2)

La precisazione riguarda il **blocco più esterno** della query di definizione.

Vista non aggiornabile (join esplicito nel blocco esterno):

```
CREATE VIEW ImpBO(CodImp,Nome,Sede,Ruolo,Stipendio)
AS SELECT I.*
   FROM Imp I JOIN Sedi S ON (I.Sede = S.Sede)
   WHERE S.Citta = 'Bologna'
```

Vista aggiornabile (join spostato in subquery):

```
CREATE VIEW ImpBO(CodImp,Nome,Sede,Ruolo,Stipendio)
AS SELECT I.*
   FROM Imp I
   WHERE I.Sede IN (SELECT S.Sede
                    FROM Sedi S
                    WHERE S.Citta = 'Bologna')
```

Le due viste sono **logicamente equivalenti**, ma solo la seconda è aggiornabile.

Viste con CHECK OPTION (1)

Per le viste aggiornabili si presenta un ulteriore problema. Si consideri:

```
INSERT INTO ImpBO(CodImp,Nome,Sede,Ruolo,Stipendio)
VALUES ('E009','Azzurri','S03','Analista',2000)
```

Problema

Il valore di Sede ('S03') non rispetta la specifica della vista ImpBO (solo impiegati di Bologna). Una successiva query su ImpBO **non restituirebbe** la tupla appena inserita!

Soluzione

All'atto della definizione della vista si può specificare la clausola WITH CHECK OPTION, che impone che ogni tupla inserita nella vista sia anche **restituita dalla sua query di definizione**.

Viste con CHECK OPTION (2)

```
CREATE VIEW ImpBO(CodImp,Nome,Sede,Ruolo,Stipendio)
AS SELECT I.*
   FROM Imp I
   WHERE I.Sede IN (SELECT S.Sede
                    FROM Sedi S
                    WHERE S.Citta = 'Bologna')
WITH CHECK OPTION
```

In questo modo l'inserimento:

```
INSERT INTO ImpBO(CodImp,Nome,Sede,Ruolo,Stipendio)
VALUES ('E009','Azzurri','S03','Analista',2000)
```

viene **impedito** dal DBMS

Tipi di CHECK OPTION (1)

Si considera il caso in cui una vista **V1** sia definita in termini di un'altra vista **V2**.

`WITH CASCADED CHECK OPTION` (default)

Se si crea V1 con `WITH CHECK OPTION`, il DBMS verifica che la nuova tupla *t* soddisfi sia la specifica di V1 **sia quella di V2** (e di tutte le altre viste dipendenti), indipendentemente dal fatto che V2 sia stata definita con `CHECK OPTION`.

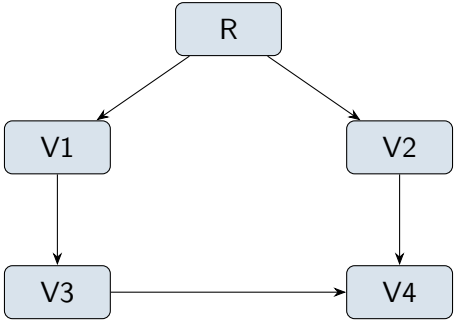
È il comportamento di default: equivalente a `WITH CASCADED CHECK OPTION`.

`WITH LOCAL CHECK OPTION`

Il DBMS verifica solo che *t* soddisfi la specifica di V1 e delle sole viste da cui V1 dipende per cui è stata **esplicitamente** specificata la `CHECK OPTION`.

Tipi di CHECK OPTION (2)

Grafo di dipendenze tra viste:



Viste controllate all'inserimento:

INSERT INTO	Si controllano...
V1	–
V2	–
V3	V1, V3
V4	V1, V2, V4

(Assumendo: V3 = LOCAL, V4 = CASCADED, V1 e V2 senza CHECK OPTION)

Table expressions (1)

Una delle caratteristiche più interessanti di SQL è la possibilità di usare all'interno della clausola FROM una **subquery** che definisce dinamicamente una tabella derivata: **table expression**.

Esempio

Per ogni sede, lo stipendio massimo e quanti impiegati lo percepiscono

```
SELECT SM.Sede, SM.MaxStip, COUNT(*) AS N
FROM Imp I,
     (SELECT Sede, MAX(Stipendio) AS MaxStip
      FROM Imp
      GROUP BY Sede) AS SM(Sede, MaxStip)
WHERE I.Sede = SM.Sede
  AND I.Stipendio = SM.MaxStip
GROUP BY SM.Sede, SM.MaxStip
```

Table expressions (2)

Le table expressions possono essere usate per realizzare **diversi passi di aggregazione**:

Esempio

Per ogni valore (arrotondato) di stipendio medio per sede, il numero delle sedi che pagano tale stipendio medio

```
SELECT AvgStip, COUNT(*) AS NumSedi
FROM (SELECT Sede, AVG(Stipendio)
      FROM Imp
      GROUP BY Sede) AS StipSedi(Sede, AvgStip)
GROUP BY AvgStip
```

⇒ Equivalente alla soluzione con le viste, ma senza definire una vista persistente.

Table expressions correlate (1)

Una table expression può essere **correlata** a un'altra tabella presente nella clausola FROM. In DB2 si usa la parola riservata TABLE.

Esempio

Per ogni sede, la somma degli stipendi pagati agli analisti

```
SELECT S.Sede, Stip.TotStip
FROM Sedi S,
     TABLE(SELECT SUM(Stipendio)
           FROM Imp I
           WHERE I.Sede = S.Sede
             AND I.Ruolo = 'Analista') AS Stip(TotStip)
```

Nota

Le sedi senza analisti **compaiono** nell'output con TotStip = NULL. Usando il GROUP BY lo stesso risultato richiederebbe un LEFT OUTER JOIN, con attenzione ai valori nulli.

Table expressions correlate (2)

Esempio

Per ogni sede, il numero di analisti e la somma degli stipendi

```
SELECT S.Sede, Stip.NumAn, Stip.TotStip
FROM Sedi S,
     TABLE(SELECT COUNT(*), SUM(Stipendio)
           FROM Imp I
           WHERE I.Sede = S.Sede
             AND I.Ruolo = 'Analista')
     AS Stip(NumAn, TotStip)
```

Per le sedi senza analisti: `NumAn = 0`, `TotStip = NULL`.

Con `LEFT OUTER JOIN` e `GROUP BY`, per le sedi senza analisti `TotStip` è nullo, ma `COUNT(*)` darebbe **1** (non 0), perché c'è una tupla nel risultato con valori nulli. Occorre usare `COUNT(CodImp)`.

Limiti delle table expressions

Esempio

La sede in cui la somma degli stipendi è massima

```
SELECT Sede
FROM (SELECT Sede, SUM(Stipendio) AS TotStip
      FROM Imp
      GROUP BY Sede) AS SediStip
WHERE TotStip = (SELECT MAX(TotStip)
                 FROM (SELECT Sede, SUM(Stipendio) AS TotStip
                       FROM Imp
                       GROUP BY Sede) AS SediStip2)
```

Problema

Benché la query sia corretta, le due table expressions sono **identiche**: la stessa subquery viene calcolata due volte, portando a inefficienza e scarsa leggibilità.

Common table expressions

L'idea alla base delle **common table expressions** (CTE) è definire una "vista temporanea" che può essere usata in una query come se fosse una VIEW.

```
WITH SediStip(Sede, TotStip)
AS (SELECT Sede, SUM(Stipendio)
    FROM Imp
    GROUP BY Sede)

SELECT Sede
FROM SediStip
WHERE TotStip = (SELECT MAX(TotStip)
                 FROM SediStip)
```

Sintassi generale

La clausola WITH supporta la definizione di **più CTE**:

```
WITH CTE1(...) AS (...),
     CTE2(...) AS (...)
SELECT ...
```

WITH e interrogazioni ricorsive (1)

Si consideri la tabella `Genitori(Figlio, Genitore)`.

Esempio

Trova tutti gli antenati (genitori, nonni, bisnonni, ...) di una persona

La query è **ricorsiva**: richiede un numero di (self-)join non noto a priori — non è esprimibile con SQL standard.

La CTE ricorsiva `Antenati(Persona, Avo)` è composta da:

- Una **subquery base** (non ricorsiva) che inizializza con i dati di `Genitori`
- Una **subquery ricorsiva** che aggiunge le coppie risultanti dal join tra `Genitori` e `Antenati`

Genitori:

Figlio	Genitore
Anna	Luca
Luca	Maria
Luca	Giorgio
Giorgio	Lucia

Antenati (risultato):

Persona	Avo
Anna	Luca
Luca	Maria
Luca	Giorgio
Giorgio	Lucia
Anna	Maria
Anna	Giorgio
Anna	Lucia
Luca	Lucia

WITH e interrogazioni ricorsive (2)

```sql
WITH Antenati(Persona, Avo)
AS (
  (SELECT Figlio, Genitore    -- subquery base
   FROM Genitori)
  UNION ALL
  (SELECT G.Figlio, A.Avo    -- subquery ricorsiva
   FROM Genitori G, Antenati A
   WHERE G.Genitore = A.Persona)
)

SELECT Avo
FROM Antenati
WHERE Persona = 'Anna'
```

Nota

L'uso di `UNION ALL` garantisce che tutte le tuple vengano propagate ad ogni iterazione, comprese le eventuali duplicate.

WITH e interrogazioni ricorsive (3) — Valutazione

Ad ogni iterazione il DBMS aggiunge ad `Antenati` le tuple risultanti dal join tra `Genitori` e le **sole tuple aggiunte** all'iterazione precedente (semantica "delta").

Iterazione 0 (base):

Anna	Luca
Luca	Maria
Luca	Giorgio
Giorgio	Lucia

Iterazione 1 (ricorsiva):

Anna	Maria
Anna	Giorgio
Luca	Lucia

Iterazione 2 (ricorsiva):

Anna	Lucia

Iterazione 3:

Nessuna nuova tupla $\Rightarrow$ **STOP**

La terminazione è garantita se il DB è **aciclico** (DAG).

Tipi notevoli di query ricorsive

Caso base (già visto)

DB aciclico (DAG): la terminazione è garantita e non si calcola nessuna informazione sui "percorsi" trovati.

Percorso: sequenza di tuple del DB che generano una tupla nel risultato finale (nell'esempio, una specifica "linea genealogica").

Due casi notevoli da esaminare:

1. **Query ricorsive con informazione sui percorsi** (lunghezza, costo cumulativo, ...)
2. **Condizione di stop** per query ricorsive su DB con **cicli**

Informazione sui percorsi (1)

Il caso più semplice prevede l'aggiunta di un'informazione sulla **lunghezza** (distanza, livello, ecc.) del percorso.

Genitori:

Figlio	Genitore
Anna	Luca
Luca	Maria
Luca	Giorgio
Giorgio	Lucia

Antenati (con lunghezza):

Persona	Avo	Lungh
Anna	Luca	1
Luca	Maria	1
Luca	Giorgio	1
Giorgio	Lucia	1
Anna	Maria	2
Anna	Giorgio	2
Luca	Lucia	2
Anna	Lucia	3

Informazione sui percorsi (2)

La lunghezza parte da **1** e si incrementa di 1 ad ogni passo ricorsivo:

```
WITH Antenati(Persona, Avo, Lungh)
AS (
  (SELECT Figlio, Genitore, 1  -- caso base
   FROM Genitori)
  UNION ALL
  (SELECT G.Figlio, A.Avo, A.Lungh + 1
   FROM Genitori G, Antenati A
   WHERE G.Genitore = A.Persona)
)

SELECT *
FROM Antenati
WHERE Persona = 'Anna'
```

Informazione sui percorsi (3) — Bill of Materials

In alcuni casi l'informazione sui percorsi è un **costo cumulativo moltiplicativo**:

Esempio: relazione PARTI

Per ogni parte si conoscono le sotto-parti e le quantità necessarie per assemblarla.
Si vuole sapere quante unità di una parte servono in totale per costruire un'altra parte.

Parti:

Parte	Subparte	Qta
P1	P2	2
P1	P3	3
P2	P4	5
P3	P4	4

Per costruire P1 occorrono:

- Percorso P1–P2–P4: $2 \times 5 = 10$ unità di P4
- Percorso P1–P3–P4: $3 \times 4 = 12$ unità di P4
- Totale P4: $10 + 12 = \mathbf{22}$

Informazione sui percorsi (4) — Schema soluzione

La soluzione prevede due fasi:

1. **Ricorsione** per calcolare l'informazione di ogni singolo percorso (prodotto delle quantità):
   - P1–P2–P4: $2 \times 5 = 10$
   - P1–P3–P4: $3 \times 4 = 12$
2. **Aggregazione** dei risultati dei singoli percorsi sulla vista ricorsiva (`GROUP BY + SUM`)

Percorsi (dopo ricorsione):

Comp.	Comp.	Qty
P1	P2	2
P1	P3	3
P2	P4	5
P3	P4	4
P1	P4	10
P1	P4	12

Risultato finale (dopo GROUP BY):

Comp.	Comp.	Qt. Totale
P1	P2	2
P1	P3	3
P2	P4	5
P3	P4	4
P1	P4	22

Informazione sui percorsi (5) — Query BOM

Ad ogni passo si **moltiplica** la quantità per quella del nodo aggiunto:

```
WITH Percorsi(Composto, Componente, Qty)
AS (
  (SELECT Parte, Subparte, Qta
   FROM Parti)
  UNION ALL
  (SELECT H.Composto, P.Subparte, H.Qty * P.Qta
   FROM Parti P, Percorsi H
   WHERE H.Componente = P.Parte)
)

SELECT Composto, Componente, SUM(Qty) AS QtTotale
FROM Percorsi
GROUP BY Composto, Componente
```

Database ciclici

Nel caso di DB **ciclici** occorre prestare particolare attenzione alla terminazione.

Rischio

Senza condizioni di stop, la ricorsione può non terminare mai!

Non disponendo di altri strumenti, occorre inserire nella subquery ricorsiva un **predicato** che sia falso per tutte le nuove tuple che si andrebbero ad aggiungere, fermando così l'iterazione.

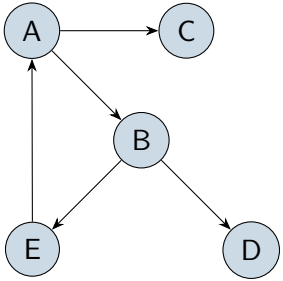
Casi tipici di condizione di stop:

- **Limitazione sulla lunghezza massima** dei percorsi
- **Limitazione sul "costo"** (somma dei pesi degli archi)

Limitazione sulla lunghezza dei percorsi (1)

Il grafo contiene un **ciclo** (ad esempio: A–B–E–A–B–E–A–. . .).

Fissando un **limite sulla lunghezza** dei percorsi si garantisce che ogni percorso trovato abbia lunghezza finita, quindi l'esecuzione prima o poi termina.



Attenzione

Il limite va scelto in modo da includere tutti i percorsi "interessanti".

Limitazione sulla lunghezza dei percorsi (2)

Con limite a **3** (e condizione anti-ciclo `P.Da <> S.To`):

```
WITH Percorsi(Da, A, Lungh)
AS (
  (SELECT From, To, 1
   FROM Siti)
  UNION ALL
  (SELECT P.Da, S.To, P.Lungh + 1
   FROM Siti S, Percorsi P
   WHERE P.A = S.From
     AND P.Da <> S.To
     AND P.Lungh + 1 <= 3)
)

SELECT *
FROM Percorsi
```

La condizione `P.Da <> S.To` serve ad evitare di includere un percorso **inutile** nel risultato (ciclo che torna al punto di partenza).

Limitazione sulla lunghezza dei percorsi (3) — Nota

La condizione `P.Da <> S.To` serve ad evitare di includere percorsi inutili nel risultato.

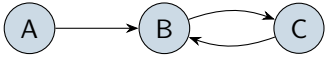
Attenzione

Benché in alcuni casi particolari possa essere usata come condizione di terminazione, **nel caso generale non è sufficiente** a garantire la terminazione in DB ciclici.

Il grafo di esempio mostra un caso in cui il ciclo è rilevabile, ma cicli più lunghi o complessi richiedono comunque il limite esplicito sulla lunghezza.

Siti:

From	To
A	B
B	C
B	C
C	B



Limitazione sul "costo"

In maniera analoga si ragiona se si vuole imporre un **costo massimo** sui percorsi:

```
WITH Percorsi(Da, A, TotKm)
AS (
  (SELECT From, To, Km
   FROM Paesi)
  UNION ALL
  (SELECT P.Da, S.To, P.TotKm + S.Km
   FROM Paesi S, Percorsi P
   WHERE P.A = S.From
     AND P.Da <> S.To
     AND P.TotKm + S.Km <= 17)
)

SELECT *
FROM Percorsi
```

Osservazione

In generale ci possono essere **più percorsi** tra 2 siti; va considerato quello a costo minore (problema del *cammino minimo*)

Riepilogo

Viste (`CREATE VIEW`)

Tabelle virtuali, interrogabili come le tabelle reali. Soggette a limiti per quanto riguarda gli **aggiornamenti**.

Table expressions

Subquery che definisce una tabella derivata, utilizzabile nella clausola `FROM` come una normale tabella.

Common table expressions (`WITH`)

"Vista temporanea" usata in una query come se fosse a tutti gli effetti una `VIEW`. Evita la ridondanza delle table expressions identiche.

Query ricorsive (`WITH RECURSIVE`)

Mediante CTE è possibile esprimere interrogazioni ricorsive, definendo "viste temporanee ricorsive" come unione di una **subquery base** e una **subquery ricorsiva**. Richiedono attenzione alla terminazione in DB ciclici